

API — Total Secure App (DT360 Core)

Proyecto: Total Secure App

Version: 1.0

Fecha: 15/09/2026

Elaborado por: Auditoria tecnica

Base: `http://<host>:3031/api` · **Alcanzable desde internet** en `http://181.198.245.50:3031` .

Verificado el **2026-09-15** con `php artisan route:list --json` contra el contenedor en produccion.

Inventario de rutas

Grupo	Cantidad
API (<code>/api/*</code>)	55
Panel web (<code>/admin/*</code>)	66
Otras (web, auth, assets)	27
Total	148

Definidas en `routes/api.php` (solo el webhook) y en los cuatro modulos:

`Modules/MobileApp/Routes/api.php` , `Modules/Acceso/` ,
`Modules/Administracion/` y `Modules/PortalApi/` .

Especificacion OpenAPI: existe y esta ACOTADA, no desactualizada

`openapi.yaml` (15 KB) documenta **7 rutas**, todas de `/api/portal/*` .

Contraste ejecutado:

Rutas en la spec que no existen en el codigo	0
Rutas en el codigo que no estan en la spec	48

Corrige la auditoria v1.0, que la clasifico como 🟡 "VERIFICAR — posiblemente desactualizada". **No lo esta**: sus 7 rutas existen y coinciden. Lo que ocurre es que **cubre un solo modulo a proposito**, y lo declara en su propia descripcion: *"API REST de solo lectura para el portal cliente (Fase 8)"*.

Reclasificada como 🟢 **VIGENTE (alcance parcial declarado)**.

La spec ademas documenta su modelo de autorizacion, incluida una decision de diseño que conviene conocer:

"pedir una institucion ajena devuelve 403, no una lista vacia: una respuesta vacia permitiria sondear que codigos de institucion existen"

Lo que falta es **la API movil**: 48 rutas sin especificacion.

Autenticacion

Middleware	Uso
ApiAuthenticate	token de Sanctum; la mayoría de las rutas
Authenticate:api	dos rutas heredadas (GET /api/acceso , GET /api/administracion)
CheckPermission:<permiso>	permiso granular por endpoint (Fase 6)

El login (POST /api/login) emite un token de **Sanctum** con expiracion (sanctum.expiracion_token_movil) y un **refresh token** de bin2hex(random_bytes(32)) .

Antes de emitirlo comprueba, en orden: que el usuario exista, que usu_state != 0 , la contraseña con Hash::check , que tenga **gestion asignada** (user_has_gestions) y que tenga **al menos un rol activo**.

● CUATRO rutas publicas — una es una toma de cuentas

Verificado en route:list : solo estas cuatro carecen de autenticacion.

Ruta	Publica por	Evaluacion
POST /api/login	necesario	✓ correcto
POST /api/solicitud_paswchg	flujo de "olvide mi contraseña"	⚠ devuelve el token en la respuesta
POST /api/procesar_paswchg	idem	● NO VALIDA NINGUN TOKEN
POST /api/whatsapp/webhook/{token}	Evolution API no tiene token de usuario	✓ bien protegido

● POST /api/procesar_paswchg — cambia la contraseña de cualquier usuario

Codigo (Modules/MobileApp/Http/Controllers/LoginController.php):

```
public function procesar_cambiopass(Request $request)
{
    $user_id    = $request->user_id;
    $password   = $request->password;
    $password2  = $request->password2;

    $rsUsuario = users::find($user_id);
    if (!$rsUsuario) { return error; }

    // ... validaciones de FORMATO de la contraseña ...

    $rsUsuario->usu_password    = $password;
    $rsUsuario->remember_token = "";
    $rsUsuario->save();
}
```

El `remember_token` que genera `solicitud_cambiopass` nunca se comprueba.

El metodo solo necesita `user_id` y la contraseña nueva.

Middleware confirmado: `['api']` . Ninguna autentificacion.

Impacto: cualquiera que alcance el endpoint —y el sistema **responde desde internet**— puede tomar control de **cualquiera de las 880 cuentas**, incluidas las administrativas, enviando:

```
POST /api/procesar_paswchg
{ "user_id": 1, "password": "Abcd1234", "password2": "Abcd1234" }
```

Los `user_id` son enteros secuenciales: no hay que adivinarlos.

Desde una cuenta administrativa se alcanzan los **12.664 registros biometricos**, 9.769 accesos y 8.667 rondas.

No se ejecuto la prueba: habria cambiado la contraseña de un usuario real en produccion. La evidencia de codigo y el middleware vacio son concluyentes.

Atenuante parcial: el modelo `users` cifra la contraseña al guardar (evento `saving` → `Hash::make`), asi que **no se almacenan en claro**. Eso limita el dano, no la toma de cuentas.

Recomendacion (lo primero de todo el proyecto):

1. **Validar el** `remember_token` contra el usuario, y que no haya expirado.
2. Generarlo con `random_bytes`, no con `rand(1000, 10000000)`.
3. **No devolverlo en la respuesta** de `solicitud_paswchg`.
4. Invalidarlo tras el primer uso.

⚠ POST /api/solicitud_paswchg entrega el token en la respuesta

```
$aleatorio = rand(1000, 100000000);  
...  
return response()->json([... , 'token' => $aleatorio, 'user_id' => $usuario->id]);
```

Aunque se corrigiera procesar_paswchg, **esto seguiria permitiendo el ataque**: se pide el cambio para una cedula cualquiera y la respuesta entrega el token y el user_id.

rand() ademas **no es criptograficamente seguro** y el rango (1000-10.000.000) es pequeño.

✅ POST /api/whatsapp/webhook/{token} — bien resuelto

```
$esperado = (string) config('avisos.whatsapp.webhook_token');  
if ($esperado === '' || !hash_equals($esperado, $token)) {  
    return response()->json(['ok' => false], 404);  
}
```

Token en la ruta, comparado con hash_equals (resistente a ataques de tiempo), y **404 si no hay token configurado**. El comentario explica el porque: *"Sin token configurado el webhook no existe: es una puerta abierta a que cualquiera simule respuestas de guardias."*

Es la implementacion de webhook mejor hecha del servidor.

Rutas por area

Todas requieren ApiAuthenticate salvo lo indicado.

Sesion y perfil

Metodo	Ruta	Acceso
POST	/api/login	🔓 publica
POST	/api/solicitud_paswchg	🔴 publica
POST	/api/procesar_paswchg	🔴 publica, sin validar token
POST	/api/seleccionar_perfil	🔑
POST	/api/procesar_perfil	🔑
POST	/api/perfil-extras	🔑

Accesos (9.769 registros)

Metodo	Ruta
GET	/api/acceso (usa Authenticate:api , middleware heredado)
POST	/api/acceso
POST	/api/acceso/preregistro · /api/acceso/preregistros
POST	/api/acceso/cancelar-preregistro
POST	/api/accesosbyinst
POST	/api/accesout

Biometria (12.664 registros)

Metodo	Ruta	Permiso
POST	/api/biometria	CheckPermission:biometria.marcar

Rondas (8.667 cabeceras)

Metodo	Ruta
POST	/api/rondas · /api/rondas_gestion
POST	/api/rondas_detalle · /api/rondas_detalle_gestion
POST	/api/rondas_detalle_qrcode

Alertas (278 registros)

Metodo	Ruta	Nota
POST	/api/alert/crear	la llama el boton EMERGENCIA de la app 1.0.2
POST	/api/alert/today · /api/alert/estadisticas	
POST	/api/alert/{id}/atender · /api/alert/{id}/cancelar	
GET	/api/alert/{id}/historial	

Inventario

Metodo	Ruta
POST	/api/inventario/listbyinst · /listsave · /finishsave
POST	/api/inventario/registrar-baja

Turnos y vacantes

Metodo	Ruta
POST	/api/turnos-del-dia · /turnos-proximos · /turnos-cumplimiento
POST	/api/turnos-avisar-ausencia · /turnos-vincular-marcaje
POST	/api/vacantes-disponibles · /vacantes-postular
POST	/api/vacantes-mis-postulaciones · /vacantes-retirar

Novedades y notificaciones

Metodo	Ruta
POST	/api/novedad_create · /api/novedad_listbydate
POST	/api/notification/user · /institution · /bulk
POST	/api/token/save · /api/token/remove (token de notificaciones push)

Instituciones

Metodo	Ruta
POST	/api/instituciones
GET	/api/administracion (Authenticate:api)

Portal cliente — la unica parte con OpenAPI

Solo lectura. Cada una exige su permiso `portal.*`, que trae el rol `Cliente`.

Metodo	Ruta
GET	/api/portal/instituciones
GET	/api/portal/resumen
GET	/api/portal/biometria
GET	/api/portal/rondas
GET	/api/portal/novedades
GET	/api/portal/accesos
GET	/api/portal/alertas

Separacion de alcances, segun la spec: el rol `Cliente` "no hereda ningun permiso de la app movil, asi que un token del portal no puede escribir en los endpoints de la app, ni un token de la app leer aqui". Es una decision correcta; **su verificacion efectiva sigue pendiente** (prueba T-30).

Fuera del alcance de esta documentacion de API: son vistas de Laravel, no endpoints REST. Cubren catalogos (países, provincias, ciudades, puestos, roles, organizaciones, instituciones, productos de inventario y plantillas).

Observaciones de diseño

Casi todo es POST. De 55 rutas, 47 son POST, incluidas consultas como `turnos-del-dia` o `vacantes-disponibles`. Funciona, pero se aparta de REST y complica el cacheo y la lectura de los logs.

Dos generaciones de middleware conviven. `Authenticate:api` en dos rutas antiguas frente a `ApiAuthenticate` en el resto. Conviene unificarlas.

Permisos granulares solo en algunas rutas. `CheckPermission` aparece en biometria y en el portal; en el resto basta el token. La autorizacion fina depende entonces del controlador: `NO DETERMINADO` cuanto valida cada uno.